The most asked RxJS interview questions focus on Observables, Subjects, operators like **switchMap**, error handling, and differences between cold vs hot observables. Recruiters often test practical knowledge of how RxJS integrates with Angular (HTTP calls, forms, event handling) and how developers manage subscriptions to avoid memory leaks.

Frequently Asked RxJS Interview Questions

Basic

What is RxJS? Reactive Extensions for JavaScript, a library for handling asynchronous data streams using Observables.
What is an Observable? A data stream that can emit multiple values over time (unlike Promises).
What are Observers? Objects with next, error, and complete callbacks to consume Observable emissions.
What is a Subscription? Represents execution of an Observable; can be cancelled with .unsubscribe().

Intermediate

Difference between Subject and BehaviorSubject?
Subject: multicast stream, no initial value.
BehaviorSubject: requires initial value, emits latest value to new subscribers.
Difference between mergeMap, switchMap, concatMap, exhaustMap?
mergeMap: runs all inner Observables concurrently.
switchMap: cancels previous Observable when a new one arrives.
concatMap: queues Observables sequentially.
exhaustMap: ignores new Observables until current completes.
What is pipe() **in RxJS?** Used to chain multiple operators for transforming streams.
What is takeUntil()**?** Completes an Observable when another emits—common unsubscribe pattern.

Advanced

- **Cold vs Hot Observables?**
  - Cold: start emitting only when subscribed (e.g., HTTP calls).
  - Hot: emit values regardless of subscriptions (e.g., mouse events).
- **How is error handling done in RxJS?** Using operators like catchError() or retry().
- **What is the difference between** ReplaySubject **and** AsyncSubject**?**
  - ReplaySubject: replays past values to new subscribers.
  - AsyncSubject: emits only the last value upon completion.
- **How do you prevent memory leaks in Angular with RxJS?** Use takeUntil(), async pipe, or manage subscriptions carefully.

Quick Comparison Table

Concept	Key Point
Observable	Emits multiple values over time
Observer	Consumes emissions via callbacks
Subscription	Controls execution; can unsubscribe

Concept	Key Point
Subject	Multicasts values, no initial value
BehaviorSubject	Requires initial value, emits latest value to new subscribers
ReplaySubject	Replays past values
AsyncSubject	Emits only last value on completion
switchMap	Cancels previous observable on new one
mergeMap	Runs all inner observables concurrently
concatMap	Queues observables sequentially
exhaustMap	Ignores new observables until current completes

Risks & Best Practices

- **Memory leaks**: Always unsubscribe or use async pipe.
- **Nested subscriptions**: Avoid; use higher-order mapping operators.
- **Confusing Subjects**: Choose BehaviorSubject when latest value is needed immediately.
- **Error handling**: Always wrap with catchError() or retry() for resilience.

since you're preparing for Angular interviews, focus especially on **operators (**switchMap**,** mergeMap**), Subjects, and unsubscribe strategies**—these are the most common areas where candidates are tested. Would you like me to create a **mock Q&A set with sample answers** so you can practice delivering concise, confident responses?

To prevent memory leaks in Angular with RxJS, you must always clean up subscriptions when components are destroyed. The most effective strategies are using the takeUntil **pattern with a** Subject**, leveraging Angular's** async **pipe in templates, and applying operators like** shareReplay **to avoid duplicate subscriptions.**

Why Memory Leaks Happen

- Angular components often subscribe to Observables (HTTP calls, route params, form changes).
- When a component is destroyed, Angular does **not automatically unsubscribe**.
- Active subscriptions keep references alive → components aren't garbage collected → memory usage grows over time.

Best Practices to Prevent Memory Leaks

1. Use takeUntil with a Destroy Subject

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.userService.getUser()
    .pipe(takeUntil(this.destroy$))
    .subscribe(user => this.user = user);
```

```
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Ensures all subscriptions end when the component is destroyed.

2. Use Angular's async Pipe in Templates

html

```
<div *ngIf="user$ | async as user">
  Hello, {{ user.name }}
</div>
```

- Angular automatically subscribes/unsubscribes.
- Cleaner, declarative, and avoids manual subscription management.

3. Use shareReplay for Shared Streams

```
user$ = this.http.get<User>('/api/user').pipe(shareReplay(1));
```

- Prevents multiple API calls when multiple subscribers exist.
- Caches and shares the response across subscribers.

4. Use finalize() for Cleanup

```
this.http.get('/api/data')
  .pipe(finalize(() => console.log('Request completed')))
  .subscribe();
```

- Useful for stopping loaders/spinners when requests end.

Quick Comparison of Approaches

Approach	Pros	Cons/Limitations
takeUntil	Works everywhere, explicit control	Requires boilerplate in each comp
async pipe	Automatic cleanup, declarative	Only works in templates
shareReplay	Prevents duplicate API calls	Must manage cache invalidation
finalize	Good for cleanup tasks (UI loaders)	Doesn't handle subscription leaks

Risks & Debugging Tips

- **Symptoms of leaks**: Duplicate API calls, console logs repeating, rising memory usage.
- **Debugging tools**: Chrome DevTools (Memory tab), Angular DevTools.
- **Best practice**: Create a **base component class** with destroy$ logic to avoid repeating code across components.

“In Angular, memory leaks happen when subscriptions aren’t cleaned up. I prevent them using the takeUntil pattern with a Subject in ngOnDestroy, or by using Angular’s async pipe for template bindings. For shared streams, I use shareReplay to avoid duplicate API calls. This ensures components are garbage collected and performance remains stable.”

Would you like me to **draft a concise interview-ready answer script** (2–3 sentences) so you can practice delivering it confidently?

Great question, Sid. For **experienced candidates**, interviewers often move beyond definitions and ask **scenario-based RxJS questions** to test how you apply concepts in real-world Angular apps. Here are the most common ones:

Scenario-Based RxJS Interview Questions

1. Search Autocomplete

- *Scenario:* You’re building a search bar that queries an API as the user types.
- *Question:* How would you prevent multiple API calls and ensure only the latest query is processed?
- *Expected Answer:* Use debounceTime() to delay requests, distinctUntilChanged() to avoid duplicates, and switchMap() to cancel previous requests.

2. Form Value Changes

- *Scenario:* You need to react to form input changes and validate them asynchronously.
- *Question:* How would you handle this efficiently?
- *Expected Answer:* Use formControl.valueChanges.pipe(debounceTime(), switchMap(...)) to trigger async validation without flooding the server.

3. Polling API Data

- *Scenario:* You must poll an API every 5 seconds until a condition is met.
- *Question:* How would you implement this?
- *Expected Answer:* Use interval(5000).pipe(switchMap(() => http.get(...)), takeWhile(...)).

4. Canceling HTTP Requests

- *Scenario:* A user navigates away from a component while an API call is in progress.
- *Question:* How do you cancel the request to avoid memory leaks?
- *Expected Answer:* Use takeUntil(destroy$) with ngOnDestroy, or switchMap() which automatically cancels previous requests.

5. Combining Multiple Streams

- *Scenario:* You need to combine user profile data from one API and permissions from another.
- *Question:* How would you merge these results?

- *Expected Answer:* Use forkJoin([http.get('profile'), http.get('permissions')]) for parallel calls, or combineLatest() if streams are continuous.

6. Error Handling

- *Scenario:* An API intermittently fails, but you want to retry 3 times before showing an error.
- *Question:* How would you implement this?
- *Expected Answer:* Use retry(3) with catchError() for graceful fallback.

7. Hot vs Cold Observables

- *Scenario:* You're tracking mouse movements and also fetching data from an API.
- *Question:* How do you explain the difference in behavior?
- *Expected Answer:* Mouse events are **hot** (emit regardless of subscribers), while HTTP calls are **cold** (start only when subscribed).

Quick Table of Scenarios

Scenario	Key Operators/Concepts
Search autocomplete	debounceTime, distinctUntilChanged, switchMap
Form validation	valueChanges, switchMap, debounceTime
API polling	interval, switchMap, takeWhile
Cancel requests	takeUntil, switchMap
Combine streams	forkJoin, combineLatest
Error handling	retry, catchError
Hot vs Cold streams	Event streams vs HTTP requests

👉 For **senior-level interviews**, they'll often ask you to **write code on the spot** for these scenarios. A strong candidate not only knows the operators but also explains **why** they chose them (e.g., switchMap cancels stale requests, forkJoin waits for all results).

Would you like me to **draft a mock coding exercise** (like a search autocomplete implementation with RxJS) so you can practice writing and explaining it under interview pressure?